

School of Computer Science and Engineering

(Computer Science & Engineering – CSE)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk – 562112 Ramanagara District, Karnataka, India

2025 – 2026

(VII Semester)

A Project On

"Smart Waste Management"

**ECOSTREAM-AI: A SOFTWARE-ONLY, INDIA-AWARE WASTE
INTELLIGENCE SYSTEM**

Submitted in partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE & ENGINEERING – CSE**

Project Team No.: CoEPCL-0216 (PCL Phase 2)

Submitted by

PRAJWAL PATIL (23BTRCT101)

NEERAV BABEL (23BTRCT088)

PRAKASH SHARMA (23BTRCT104)

MAHANTAPPA PATTED (23BTRCT066)

KRRISH JAIN (23BTRCT061)

Under the guidance of

Dr. ShivaKumar C,

Project Guide,

Department of Computer Science & Engineering – CSE,

School of Computer Science & Engineering, Faculty of Engineering & Technology,

Department of Computer Science & Engineering – CSE

School of Computer Science & Engineering, Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk – 562112 Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled "**Smart Waste Management (EcoStream-AI)**" is carried out by **Prajwal Patil (23BTRCT101)**, **Neerav Babel (23BTRCT088)**, **Prakash Sharma (23BTRCT104)**, **Mahantappa Patted (23BTRCT066)** and **Krrish Jain (23BTRCT061)**, the bona-fide students of Bachelor of Technology at the School of Computer Science and Engineering – CSE, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore, in partial fulfilment for the award of degree in Bachelor of Technology in Computer Science & Engineering – CSE, during the year 2025–2026. Project Team No.: CoEPCL-0216.

Dr. ShivaKumar C

Project Guide,

Computer Science &
Engineering – CSE,

Faculty of Engineering &
Technology,

JAIN (Deemed-to-be
University).

Date:

Dr. T. R. Mahesh

Program Head,

Computer Science &
Engineering – CSE,

Faculty of Engineering &
Technology,

JAIN (Deemed-to-be
University).

Date:

Dr. Kamlesh Tiwari

Head of Department,

Computer Science &
Engineering,

Faculty of Engineering &
Technology,

JAIN (Deemed-to-be
University).

Date:

Name of the Examiner

Signature of Examiner

1.

2.

DECLARATION

We, Prajwal Patil (23BTRCT101), Neerav Babel (23BTRCT088), Prakash Sharma (23BTRCT104), Mahantappa Patted (23BTRCT066) and Krrish Jain (23BTRCT061), students of the 7th semester B.Tech in Computer Science & Engineering – CSE at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), hereby declare that the project work titled "Smart Waste Management (EcoStream-AI)" has been carried out by us and submitted in partial fulfilment for the award of degree in Bachelor of Technology in Computer Science & Engineering – CSE during the academic year 2025–2026. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

PRAJWAL PATIL: 23BTRCT101

NEERAV BABEL: 23BTRCT088

PRAKASH SHARMA: 23BTRCT104

MAHANTAPPA PATTED: 23BTRCT066

KRRISH JAIN: 23BTRCT061

Place: Bangalore

Date:

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed-to-be University) for providing us with a great opportunity to pursue our Bachelor's Degree in this institution.

We are deeply thankful to several individuals whose invaluable contributions have made this project a reality. We wish to extend our heartfelt gratitude to Dr. Chenraj Roy Chand, Chancellor, for his tireless commitment to fostering excellence in teaching and research at JAIN (Deemed-to-be University). We are also profoundly grateful to the honourable Dr. Dinesh Nilkant, Pro Vice Chancellor, for his unwavering support. Furthermore, we would like to express our sincere thanks to Dr. Jitendra Kumar Mishra, Registrar, whose guidance has imparted invaluable qualities and skills that will serve us well in our future endeavours.

We extend our sincere gratitude to Dr. Kamlesh Tiwari, Head of the Department of the School of Computer Science & Engineering within the Faculty of Engineering & Technology, for his constant encouragement and expert advice. Additionally, we would like to express our appreciation to Dr. Deepak Kumar Sinha, Deputy Director (Students & Industry Relations), for his invaluable contributions and support throughout this project.

It is a matter of immense pleasure to express our sincere thanks to Dr. T. R. Mahesh, Program Head, Computer Science & Engineering – CSE, School of Computer Science & Engineering, Faculty of Engineering & Technology, for providing the right academic guidance that made our task possible.

We would like to thank our guide Dr. ShivaKumar C, Department of Computer Science & Engineering – CSE, for sparing his valuable time to extend help in every step of our work, which paved the way for the smooth progress and fruitful culmination of the project.

We would like to thank our Project Coordinator Dr. C. R. Raman and all the staff members of Computer Science & Engineering – CSE for their support. We are also grateful to our families and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in completing the work successfully.

Signature of Student(s)

PRAJWAL PATIL 23BTRCT101

NEERAV BABEL 23BTRCT088

PRAKASH SHARMA 23BTRCT104

MAHANTAPPA PATTED 23BTRCT066

KRRISH JAIN 23BTRCT061

TABLE OF CONTENTS

Chapter No.	Chapter Title	Page No.
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract	9
Chapter 1	Introduction	10
1.1	Background & Motivation	10
1.2	Objectives	11
1.3	Benefits of Research	12
Chapter 2	Literature Survey	13
2.1	Literature Review	13
2.1.1	Analysis of Literature	14
2.2	Problem Formulation and Proposed Work	14
2.2.1	Introduction	14
2.2.2	Problem Statement	15
2.2.3	System Architecture / Model	15
2.2.4	Proposed Algorithms	15
2.2.5	Proposed Work	16
Chapter 3	System Analysis and Proposed Methodology	17
3.1	Introduction	17
3.2	Problem Statement	17
3.3	System Architecture / Model	17
3.4	Proposed Algorithms	18
3.4.1	Vision Module: YOLOv11n-seg	18
3.4.2	Waste Urgency Index (WUI)	18
3.4.3	Time-Decayed Urgency-Weighted KDE	19
3.4.4	Festival-Aware Gradient Boosting Regressor	19
3.4.5	RAG-Grounded Disposal Advice	20
3.5	Proposed Work	20
Chapter 4	System Design and Implementation	21
4.1	Introduction	21
4.2	System Design Principles	21
4.3	System Architecture	21
4.4	Dataset Design & Preprocessing	23
4.4.1	Dataset Structure	23
4.4.2	Preprocessing Techniques	23
4.4.3	Data Augmentation	24
4.5	Model Architecture & Implementation	24
4.5.1	YOLOv11n-seg Backbone	24

4.5.2	Custom Classification Head	24
4.5.3	Training Mechanism	24
4.5.4	Training Callbacks	25
4.6	Web Application Design	25
4.6.1	User Interface Modules	25
4.6.2	Real-Time Prediction Flow	25
4.6.3	Error Handling	26
4.7	Implementation Workflow	27
Chapter 5	Tools and Technologies Required	28
5.1	Development Tools	28
5.2	Frontend Technologies	28
5.3	Backend Technologies	28
5.4	Deployment & Hosting Tools	29
5.5	Testing & Monitoring Tools	29
5.6	Optional / Future Integration Tools	30
Chapter 6	Software Requirement Specification (SRS)	31
6.1	Specific Requirements	31
6.2	Functional Requirements	31
6.3	Non-Functional Requirements	32
6.4	Human-Centered Design Requirements	32
6.5	Specific Requirements	33
6.5.1	Software Interfaces	33
6.5.2	Communication Interfaces	33
Chapter 7	Results and Discussion	34
7.1	Introduction	34
7.2	Model Performance Metrics	34
7.2.1	Accuracy	34
7.2.2	Precision and Recall	35
7.3	Classification Report Analysis	36
7.4	Confusion Matrix Discussion	36
7.5	Visualisation & Evaluation Insights	37
7.6	Real-Time Deployment Results	39
7.6.1	End-to-End Latency Under 50 Concurrent Users	39
7.6.2	RAG Quality (RAGAS)	40
7.6.3	System Configuration in Force	41
7.7	Discussion of Findings	41
Chapter 8	Conclusions and Future Scope	42
8.1	Conclusions	42
8.2	Future Scope	42
8.3	Summary	43
	References	44

LIST OF FIGURES

S.no	Fig Number	Page Number(s)
1	<i>Fig-4.1 Dataset Characterization</i>	22
2	<i>Fig-4.2 EcoStream AI System Architecture Diagram</i>	23
3	<i>Fig-4.3 Waste Scan Request Workflow</i>	26
4	<i>Fig-7.1 Training Curves Over 100 Epochs</i>	34
5	<i>Fig-7.2 Confidence Curve</i>	35
6	<i>Fig-7.3 Normalized Confusion Matrix(epoch 100)</i>	37
7	<i>Fig-7.4 Precision-Recall Curve for Detection Head</i>	38
8	<i>Fig-7.5 Precision-Recall Curve for Segmentation Head</i>	38
9	<i>Fig-7.76 Fig-7.9 Qualitative Segmentation Prediction on Held out Validation Batch</i>	40
10	<i>Fig-7.7 Latency Distribution under 50 Concurrent Users</i>	41
11	<i>Fig-7.8 RAGAS Evaluation Summary (Faithfulness 0.9222,)</i>	42

LIST OF TABLES

S.no	Table Number and Name	Page Number(s)
1	<i>Table-2.1 Literature Review — selected prior work relevant to EcoStream AI</i>	13
2	<i>Table-3.1 India-specific hazard weights $w(m)$ for the WUI (MoEFCC SWM Rules 2016)</i>	18
3	<i>Table-3.2 The 10-feature vector for the GBR hotspot predictor</i>	19
4	<i>Table-3.3 India festival multipliers (17 events tabulated for 2024–2026)</i>	19
5	<i>Table-4.1 YOLOv11n-seg training configuration (args.yaml)</i>	24
6	<i>Table-7.1 YOLOv11n-seg final-epoch validation metrics on the held-out 6-class split</i>	34
7	<i>Table-7.2 Per-class diagnostics (validation split) — Recall and Peak F1</i>	36
8	<i>Table-7.3 Latency under 50 concurrent POST /analyze requests</i>	39
9	<i>Table-7.4 RAG quality metrics (RAGAS) against the held-out ground-truth set</i>	40
10	<i>Table-7.5 RAG configuration in force during the RAGAS benchmark</i>	41

ABSTRACT

Municipal Solid Waste (MSW) management in dense Indian cities is constrained by two structural failures: heaps are reported by volume rather than by hazard, and collection routes are reactive rather than predictive. India generated an estimated 62 million tonnes of MSW in 2023, of which less than 25% is scientifically processed; festival peaks routinely overwhelm Urban Local Bodies (ULBs).

The proposed system, EcoStream AI, is an end-to-end software-only pipeline that converts a single citizen-captured smartphone image into (i) a segmented material breakdown using a fine-tuned YOLOv11n-seg deep learning model, (ii) a calibrated Waste Urgency Index (WUI) that weights detections by India-specific hazard categories derived from the MoEFCC Solid Waste Management Rules 2016, (iii) an urgency-weighted, time-decayed Gaussian Kernel Density Estimation (KDE) heatmap of live hotspots, and (iv) a festival-aware Gradient Boosting Regressor (GBR) that forecasts the next day's hotspots from a 10-feature vector spanning lag counts, urgency rolling means and 17 tabulated Indian festival multipliers. A Retrieval-Augmented Generation (RAG) layer grounded on a curated municipal-policy corpus returns disposal advice with measured faithfulness 0.9222 and answer relevancy 0.8607 on the RAGAS benchmark.

The deployed YOLOv11n-seg model, trained for 100 epochs on an augmented 6-class corpus on a single consumer-grade NVIDIA RTX 3050 (4 GB), reaches mAP@50–95 of 0.5963 for detection and 0.5019 for segmentation, with peak F1 of 0.65 at confidence 0.410. Under a 50-concurrent-request stress test the full end-to-end pipeline serves each request in 978.86 ms on average (P95 = 966 ms), comfortably below our 3,000 ms KPI. The complete system is implemented in Python (FastAPI, Ultralytics, scikit-learn, scipy), React (Vite) and PostgreSQL, deployed via Docker Compose, and incorporates Explainable-AI cues through the WUI breakdown and KDE visualisation. To the best of our knowledge, EcoStream AI is the first published architecture that couples a hazard-weighted urgency score with a festival-aware spatio-temporal predictor for the Indian municipal context.

Chapter 1

Introduction

Municipal Solid Waste (MSW) management is one of the most pressing urban-infrastructure challenges in India. The Central Pollution Control Board (CPCB) reports that Indian cities generated approximately 62 million tonnes of municipal solid waste in 2023, of which less than 25% is scientifically processed; the remainder is either land-filled in unregulated dumps, openly burned, or accumulates in streetside hotspots that become public-health hazards. Two structural failures dominate every reactive municipal pipeline:

1. **Hazard blindness.** Collection priority is decided by volume, not by what is in the heap. A bag of cardboard and a pile of biomedical or sanitary waste are treated identically, even though their public-health risk profiles differ by an order of magnitude.
2. **Reactive scheduling.** Routes respond to yesterday's reports rather than anticipating tomorrow's spikes. As a result, every Diwali, Holi, Ganesh Chaturthi and Onam produces a predictable backlog that no current production system mitigates.

The EcoStream AI project addresses both failures with a deliberately software-only system that runs on commodity hardware — a single 4 GB consumer GPU for training and ordinary citizen smartphones for capture — avoiding the cost and maintenance overhead of bin-mounted IoT sensors that have repeatedly failed to scale in Indian municipal pilots.

1.1 Background & Motivation

The traditional MSW collection cycle in Indian cities is a manual, schedule-based operation: trucks visit fixed routes at fixed times, collect whatever is present, and mark the route 'done'. Three empirical observations make this approach fundamentally inefficient:

- **Spatial heterogeneity is high and bursty.** Hotspots cluster around markets, railway stations, religious gathering points and informal settlements, and shift on a daily cycle. A static schedule under-serves bursty hotspots and over-serves consistently empty stretches.
- **Temporal heterogeneity is festival-driven.** Indian urban MSW generation is not a smooth function of population density; it spikes by 1.5–2.0× around major festivals,

with the shape of the spike (sweets and oil from Diwali, gulal and water-balloons from Holi, idols and floral waste from Ganesh Chaturthi) varying by event. Schedules built on rolling 7-day averages systematically miss every spike.

- **Material composition matters more than total volume.** A small amount of biomedical, e-waste or sanitary waste is more urgent than a large amount of inert C&D rubble, both because of direct hazard and because of cross-contamination of recyclable streams. The MoEFCC Solid Waste Management Rules 2016 codify a 16-category hazard hierarchy, but no current production system surfaces this hierarchy to the operator at decision time.

Smartphones, by contrast, are essentially universal in urban India and can serve simultaneously as the sensor (camera + GPS), the input device (citizen-reported scans) and the dashboard (operator alerts). A software-only pipeline that fuses these three roles is therefore a natural fit, provided that (a) the on-device latency budget is respected, (b) the model is small enough to train on accessible hardware, and (c) the language model used for disposal advice is grounded on auditable municipal policy rather than open-domain web text. EcoStream AI is designed against precisely these constraints.

1.2 Objectives

The objectives of this project, as scoped against the constraints above, are:

3. Build a containerised microservice stack that accepts a citizen-captured image plus GPS coordinates and returns, within the user's 3,000 ms KPI, a structured analysis of the waste heap including detected materials, disposal advice and a single-scalar urgency score.
4. Fine-tune a YOLOv11n-seg instance-segmentation model on an augmented 6-class waste corpus (plastic, metal, food_waste, paper, glass, cardboard) using a single consumer GPU (RTX 3050, 4 GB) and report calibrated mAP, precision, recall and F1.
5. Define and implement a Waste Urgency Index grounded in the MoEFCC SWM Rules 2016 hazard hierarchy, mapping per-detection (label, confidence) pairs onto a single scalar in $[0,1]$ that downstream collection scheduling can rank on.
6. Implement an urgency-weighted, time-decayed Gaussian KDE hotspot heatmap that automatically dims as scans age (half-life ≈ 34.66 h) and emphasises hazardous categories.

7. Implement a festival-aware Gradient Boosting Regressor that consumes a 10-dimensional feature vector spanning lag counts, urgency rolling means, neighbourhood density, day-of-week cyclic encoding and a 17-festival multiplier table to forecast tomorrow's hotspots.
8. Build a Retrieval-Augmented Generation (RAG) layer over a curated municipal-policy corpus and validate it with the RAGAS benchmark (faithfulness, answer-relevancy) against a 0.85 production threshold.
9. Deliver a React-based dual-portal frontend (Citizen + Government Dashboard) and validate the full stack under 50 concurrent users.

1.3 Benefits of Research

- **Urban Local Bodies (ULBs).** The system replaces volume counting with a hazard-aware, spatially explicit, calendar-aware decision support layer that can be adopted without buying new hardware. The latency budget (sub-1 s end-to-end) makes it compatible with on-call operator workflows.
- **Citizens.** A two-tap report (capture + submit) returns auditable disposal advice in under a second, raising the practical utility of citizen reporting beyond the 'send-to-the-void' experience of current grievance portals.
- **Sanitation workers.** Because the WUI surfaces hazardous classes (biomedical, e-waste, sanitary) explicitly, workers can arrive prepared with the correct PPE and segregation containers, improving occupational safety.
- **Policy researchers.** The festival multiplier table and the WUI weight schedule are externally auditable artifacts that can be challenged, refined and extended; the entire pipeline (data, model, code, weights) is reproducible from the released repository.

The remainder of this report is organised as follows. Chapter 2 surveys the literature on vision-based waste classification, spatio-temporal forecasting for urban events, and RAG. Chapter 3 formalises the problem and presents the proposed methodology. Chapter 4 details system design and implementation. Chapter 5 enumerates tools and technologies. Chapter 6 captures the Software Requirements Specification. Chapter 7 reports the evaluation, including vision metrics, latency, RAG quality and per-class diagnostics. Chapter 8 concludes and outlines future work.

Chapter 2

Literature Survey

2.1 Literature Review

Vision-based waste analysis sits at the intersection of three mature research streams: object detection / instance segmentation, spatio-temporal density estimation, and retrieval-augmented language generation. Table 2.1 summarises the most relevant prior art reviewed for this project.

Table 2.1: Literature Review — selected prior work relevant to EcoStream AI.

Author / Year	Domain	Method	Key Finding / Limitation
Proenca & Simoes, 2020	Waste detection dataset	TACO, 60 categories, in-the-wild	Establishes the in-the-wild benchmark; uneven class balance and biomedical scarcity directly motivate our augmentation strategy and the staged 20-class roadmap (Sec. 8.2).
Yang & Thung, 2017	Recyclable classification	TrashNet, 6 classes, lab imagery	Originating 6-class taxonomy that we adopt for the deployed model; lab imagery does not transfer well to streetside scans, motivating heavy augmentation.
Majchrowska et al., 2022	Waste detection	Deep learning over urban + natural waste	Demonstrates fine-tuned object detectors transfer to messy, in-the-wild scenes; reports significant per-class variance, which our results corroborate (Sec. 7.3).
Ultralytics, 2024	Object detection toolkit	YOLOv11 family, n / s variants	The nano variant (~2.6M params, segmentation head) fits a 4 GB GPU with batch 8 at 640 px, which drives our backbone choice.
Kannangara et al., 2018	MSW forecasting	ML on aggregate municipal tonnage (Canada)	Aggregate-level forecasting; does not ingest per-scan hazard signals or condition on festival calendars. We extend this to per-cell, per-day predictions.
Abdallah et al., 2020	MSW / AI review	Systematic review of AI in solid-waste	Catalogues a literature gap: virtually no published system couples per-incident hazard scoring with calendar-aware forecasting. EcoStream AI is positioned in this gap.

Scott, 1992	Density estimation	Multivariate KDE, Scott's bandwidth	Standard reference for nonparametric spatial density; we use Scott's rule for h in our weighted KDE (Eq. 7).
Friedman, 2001	Regression	Gradient Boosting Regressor	Theoretical foundation for our online-trained GBR predictor (Sec. 3.4.4).
Lewis et al., 2020	NLP	Retrieval-Augmented Generation	RAG architecture used to ground disposal advice on the curated municipal-policy corpus; eliminates open-web hallucination.
Es et al., 2024	RAG evaluation	RAGAS: faithfulness, answer relevancy	Adopted unchanged as our RAG quality benchmark; Sec. 7.6 reports the measured scores.

2.1.1 Analysis of Literature

Three gaps emerge from the literature reviewed in Table 2.1.

Gap 1 — Volume vs. hazard. Existing vision pipelines report counts of detected objects per class. None convert these detections into a single, decision-ready scalar that ranks heaps by hazard. We close this gap with the Waste Urgency Index (Sec. 3.4.2), which projects (label, confidence) vectors onto $[0,1]$ via an MoEFCC-grounded weight table.

Gap 2 — Calendar-blind forecasting. Aggregate forecasting work does not condition on local festival calendars. For Indian cities this is a first-order miss: the difference between a normal Tuesday and Diwali eve in the same neighbourhood can be $2\times$ in waste generation. We close this gap with a 17-event festival multiplier table consumed by the GBR predictor (Sec. 3.4.4).

Gap 3 — Stale heatmaps. Standard KDE weights every sample equally regardless of its age. For a hotspot heatmap that should reflect the current operational picture, this is wrong: a 5-day-old scan should not look like one taken an hour ago. We close this gap with an exponential temporal decay (half-life ≈ 34.66 h) multiplied by the WUI of each scan as the per-sample KDE weight (Eq. 7).

2.2 Problem Formulation and Proposed Work

2.2.1 Introduction

The literature review above motivates a system that simultaneously (i) detects and segments waste materials in citizen imagery, (ii) scores each scan by hazard, (iii) aggregates active scans into a hazard-aware live heatmap, (iv) predicts tomorrow's hotspots calendar-aware,

and (v) returns natural-language disposal advice grounded on policy. None of the surveyed prior systems jointly addresses all five.

2.2.2 Problem Statement

Given a citizen-captured smartphone image I at GPS coordinates (ϕ, λ) and a historical scan corpus $D = \{(I_j, \phi_j, \lambda_j, t_j, u_j, m_j)\}$ with timestamps t_j , urgency scores u_j and material lists m_j , compute, within a 3,000 ms wall-clock budget under 50 concurrent users:

10. the segmented material list $\hat{m}$ of I ,
11. the scalar urgency score $\hat{u} \in [0,1]$,
12. the live heatmap $\hat{H}$ over the city's active hotspots,
13. next-day per-cell hotspot intensities $\hat{Y}$, and
14. the natural-language disposal advice $\hat{a}$ for $\hat{m}$, grounded on the municipal-policy corpus.

2.2.3 System Architecture / Model

The proposed solution is a five-service Docker Compose stack. The mobile/web client posts a multipart POST /analyze containing image bytes and GPS coordinates. The backend fans out to (a) the YOLOv11n-seg vision microservice (port 8001), (b) a Groq Llama-4 Scout multimodal classifier acting as a tie-breaker, (c) the WUI scorer, and (d) the RAG-grounded disposal-advice generator. Persistence is in PostgreSQL, which doubles as the training corpus for the GBR predictor. The analytics microservice (sharing the backend container) exposes GET /heatmap (urgency-weighted KDE) and GET /predict (festival-aware GBR).

2.2.4 Proposed Algorithms

- **YOLOv11n-seg** for instance segmentation of waste materials. Chosen for its ~2.6M-parameter footprint, native ONNX export and proven on-edge throughput.
- **Llama-4 Scout (multimodal)** via the Groq inference API as a tie-breaker for ambiguous YOLO labels.
- **Waste Urgency Index (WUI)** — a logistic hazard-weighted aggregator over per-object detections.
- **Time-decayed urgency-weighted Gaussian KDE** — `scipy.stats.gaussian_kde` with per-sample weights and Scott's-rule bandwidth.

- **Festival-aware Gradient Boosting Regressor (GBR)** — `sklearn.ensemble.GradientBoostingRegressor` trained online on the trailing 14-day scan history with 10 engineered features.
- **Retrieval-Augmented Generation (RAG)** — ChromaDB + BAAI/bge-base-en-v1.5 embeddings + Groq Llama-3 generation.

2.2.5 Proposed Work

The work splits into the following deliverables, each of which is realised in the released codebase:

15. Acquire and curate a 6-class waste image corpus, augment it programmatically (`dataset/gan_mix.py`), and train YOLOv11n-seg for 100 epochs on a single RTX 3050.
16. Implement `backend/services/urgency.py` with a 16-row MoEFCC-derived weight table; expose it via the WUI in POST `/analyze`.
17. Add an `urgency_score` column to the PostgreSQL schema (idempotent migration in `backend/db/init.sql`).
18. Re-implement `analytics/kde.py` on top of `scipy.stats.gaussian_kde` with per-sample $w_i = u_i \cdot \exp(-\lambda \Delta t_i)$ weights.
19. Re-implement `analytics/predict.py` as an online-trained `GradientBoostingRegressor` consuming a 10-dimensional feature vector that includes a 17-event festival multiplier table.
20. Build a curated municipal-policy corpus, ingest it into ChromaDB with BAAI/bge-base-en-v1.5 embeddings, and validate the answer generator with the RAGAS benchmark.
21. Deliver a React (Vite) frontend with Citizen Portal and Government Dashboard pages; integrate the heatmap and predictive layer as Leaflet overlays.
22. Execute a 50-concurrent-request stress test and confirm the sub-3 s end-to-end latency KPI.

Chapter 3

System Analysis and Proposed Methodology

3.1 Introduction

This chapter formalises the methodology that turns the raw inputs (image, GPS, history) into the five outputs enumerated in Section 2.2.2. We first state the formal problem (Section 3.2), then describe the system model (Section 3.3), then specify each algorithm in mathematical detail (Section 3.4), and finally close with the implementation roadmap (Section 3.5).

3.2 Problem Statement

Let $I \in \mathbb{R}^{H \times W \times 3}$ be the input RGB image, $(\phi, \lambda) \in \mathbb{R}^2$ the GPS pair, and $D_T = \{(\phi_j, \lambda_j, t_j, u_j) : t-T \leq t_j \leq t\}$ the trailing-T-day scan history (we use $T = 14$). Define the per-cell spatial discretisation $G(\phi, \lambda)$ at 0.005° resolution (≈ 500 m at Indian latitudes). The system must produce, within a 3,000 ms latency budget:

- $\hat{m} = \text{Vision}(I)$ (material list)
- $\hat{u} = \text{WUI}(\hat{m}, c)$ (urgency score)
- $\hat{H}(x) = \text{KDE}_\lambda(x; D_T)$ (live heatmap density)
- $\hat{Y}(c) = \text{GBR}_\theta(\phi(c, t+1))$ (per-cell next-day forecast)
- $\hat{a} = \text{LLM}(\hat{m}, \text{Retr}(\hat{m}))$ (grounded disposal advice)

where c is the per-detection confidence vector, M is the 6-class material vocabulary (extensible to 16 in the WUI table), $\phi(c, t+1) \in \mathbb{R}^{10}$ is the GBR feature vector for cell c on tomorrow's date, and Retr is a top- k chunk retriever over the municipal-policy corpus ($k = 3$, chunk size 500, overlap 50).

3.3 System Architecture / Model

The system architecture is a containerised five-service stack. All inter-service traffic is HTTP/JSON; data persistence is in PostgreSQL; document persistence (RAG corpus) is in ChromaDB. The end-to-end critical path of a single POST /analyze call is:

23. Receive multipart payload (image bytes + GPS + optional city).

24. Vision call (vision:8001/detect) returns the labels-and-confidence payload (median ~489 ms; see Sec. 7.6).
25. Vision tie-break via Groq Llama-4 Scout when YOLO confidence is ambiguous.
26. WUI scoring on the merged label list (Sec. 3.4.2).
27. RAG retrieval and generation of the disposal advice (~376 ms).
28. Database insert + post-update (~16 ms).
29. Return the structured JSON response, which the frontend uses to update both the citizen view and the operator dashboard.

3.4 Proposed Algorithms

3.4.1 Vision Module: YOLOv11n-seg

We fine-tune YOLOv11n-seg on an augmented 6-class waste corpus at 640×640, batch 8, AdamW, 100 epochs on a single RTX 3050 (4 GB VRAM). The model is scored with the standard COCO metrics — precision, recall, IoU and the mAP averaged over IoU thresholds $\{0.50, 0.55, \dots, 0.95\}$:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}), \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{IoU}(A, B) = |A \cap B| / |A \cup B|$$

$$\text{mAP@50-95} = (1/|T|) \sum_{\tau \in T} \text{AP}(\tau)$$

3.4.2 Waste Urgency Index (WUI)

A scan with n detected objects is summarised as a single scalar in $[0,1]$ (Equation 6, paper headline equation):

$$\text{WUI} = \sigma((1 / N_{\text{max}}) \cdot \sum_i c_i \cdot w(m_i))$$

where $c_i \in [0,1]$ is the per-detection confidence, $w(m_i)$ is the MoEFCC-derived hazard weight (Table 3.1), $N_{\text{max}} = 8.0$ is a soft normaliser preventing one large heap from saturating, and $\sigma(x) = 1/(1+e^{-k(x-x_0)})$ is a re-centred logistic with $k = 6$ and $x_0 = 0.5$.

Table 3.1: India-specific hazard weights $w(m)$ for the WUI (MoEFCC SWM Rules 2016). Top block: deployed 6-class model. Bottom block: 20-class roadmap.

Material	$w(m)$	Material	$w(m)$
plastic	1.5	food_waste	2.5
metal	0.8	paper	0.6
glass	1.2	cardboard	0.5
biomedical	5.0	sanitary	4.0
e-waste	4.5	mixed	2.0
hazardous-chem	4.5	textile	1.8

rubber	1.0	wood	0.9
garden	0.4	inert/C&D	0.3

3.4.3 Time-Decayed Urgency-Weighted KDE

Given trailing-window scans $\{(x_i, t_i, u_i)\}$, the live-hotspot density at query point x is (Equation 7, paper headline equation):

$$\hat{f}(x) = (1 / N) \cdot \sum_i w_i \cdot K_h(x - x_i), \quad w_i = u_i \cdot \exp(-\lambda \cdot \Delta t_i)$$

where Δt_i is the age of scan i in hours, $\lambda = 0.02 \text{ hr}^{-1}$ (half-life $t_{1/2} = \ln 2 / \lambda \approx 34.66 \text{ h}$), K_h is the Gaussian kernel and h is Scott's-rule bandwidth $h = n^{-1/(d+4)} \cdot \sigma$ with $d = 2$.

3.4.4 Festival-Aware Gradient Boosting Regressor

For each grid cell c and target day t we construct a 10-dimensional feature vector $\phi(c, t) \in \mathbb{R}^{10}$ (Table 3.2), fit $\hat{y}(c, t) = \text{GBR}_{\theta}(\phi(c, t))$ on the (ϕ, y) pairs from the past 14 days, and predict $\hat{y}(c, t_{\text{tomorrow}})$ per cell. Hyper-parameters: 60 trees, max-depth 3, learning rate 0.08, squared-error loss.

Table 3.2: The 10-feature vector for the GBR hotspot predictor.

#	Feature	Description
1	lag_1d_count	Scans in cell on day $t-1$
2	lag_7d_count	Scans in cell on day $t-7$
3	rolling_mean_3d	Mean count over $[t-3, t-1]$
4	rolling_std_7d	Standard deviation over $[t-7, t-1]$
5	cumulative_14d	Total scans over $[t-14, t-1]$
6	mean_urgency_7d	Mean WUI in cell over $[t-7, t-1]$
7	neighbour_density_7d	8-neighbour cells, summed over 7 d
8	dow_sin	$\sin(2\pi \cdot \text{dow} / 7)$
9	dow_cos	$\cos(2\pi \cdot \text{dow} / 7)$
10	festival_multiplier	India festival bump (Table 3.3)

The festival multiplier is non-zero on ± 1 days around any of 17 tabulated Indian festivals (Table 3.3). Eve and post-day multipliers are softened to $1 + 0.6(m - 1)$ where m is the on-day multiplier.

Table 3.3: India festival multipliers (excerpt; 17 events tabulated for 2024–2026).

Festival	m	Festival	m
Diwali	2.0	Holi	1.8
Ganesh Chaturthi	1.6	Navratri	1.5
Durga Puja	1.5	Onam	1.4
Eid-ul-Fitr	1.4	Eid-ul-Adha	1.4
Dussehra	1.4	New Year	1.4

Janmashtami	1.3	Christmas	1.3
Pongal	1.3	Republic Day	1.2
Independence Day	1.2	Raksha Bandhan	1.2
Karva Chauth	1.1	—	—

3.4.5 RAG-Grounded Disposal Advice

Material labels and resolved city are projected into a query that retrieves $k = 3$ chunks (size 500, overlap 50) from the curated municipal-policy corpus indexed in ChromaDB with BAAI/bge-base-en-v1.5 embeddings. The retrieved context is assembled into a system prompt that constrains the language model to cite-only generation, the quality of which is quantified in Section 7.6.

3.5 Proposed Work

The proposed methodology decomposes into eight implementation deliverables, each independently testable:

30. **Vision module.** Train and evaluate YOLOv11n-seg on the augmented 6-class corpus.
31. **Vision tie-break.** Wire the Groq Llama-4 Scout multimodal classifier into the analyse pipeline.
32. **WUI scorer.** Implement the WUI in backend/services/urgency.py, persist the score in PostgreSQL, return it in the API response.
33. **Time-decayed KDE.** Implement the urgency-weighted KDE in analytics/kde.py on top of scipy.stats.gaussian_kde.
34. **GBR predictor.** Implement the festival-aware predictor in analytics/predict.py with the 10-feature vector and the 17-event festival table.
35. **RAG advice.** Curate the municipal-policy corpus, ingest it into ChromaDB and ground the disposal-advice prompt.
36. **Frontend.** Deliver a React (Vite) two-portal UI (Citizen + Government).
37. **Validation.** 50-concurrent-request stress test, RAGAS benchmarking, qualitative ablation of the urgency-weighted heatmap.

Chapter 4

System Design and Implementation

4.1 Introduction

This chapter walks through the realised system: design principles, service-level architecture, dataset curation, model architecture, front-end, and end-to-end implementation workflow.

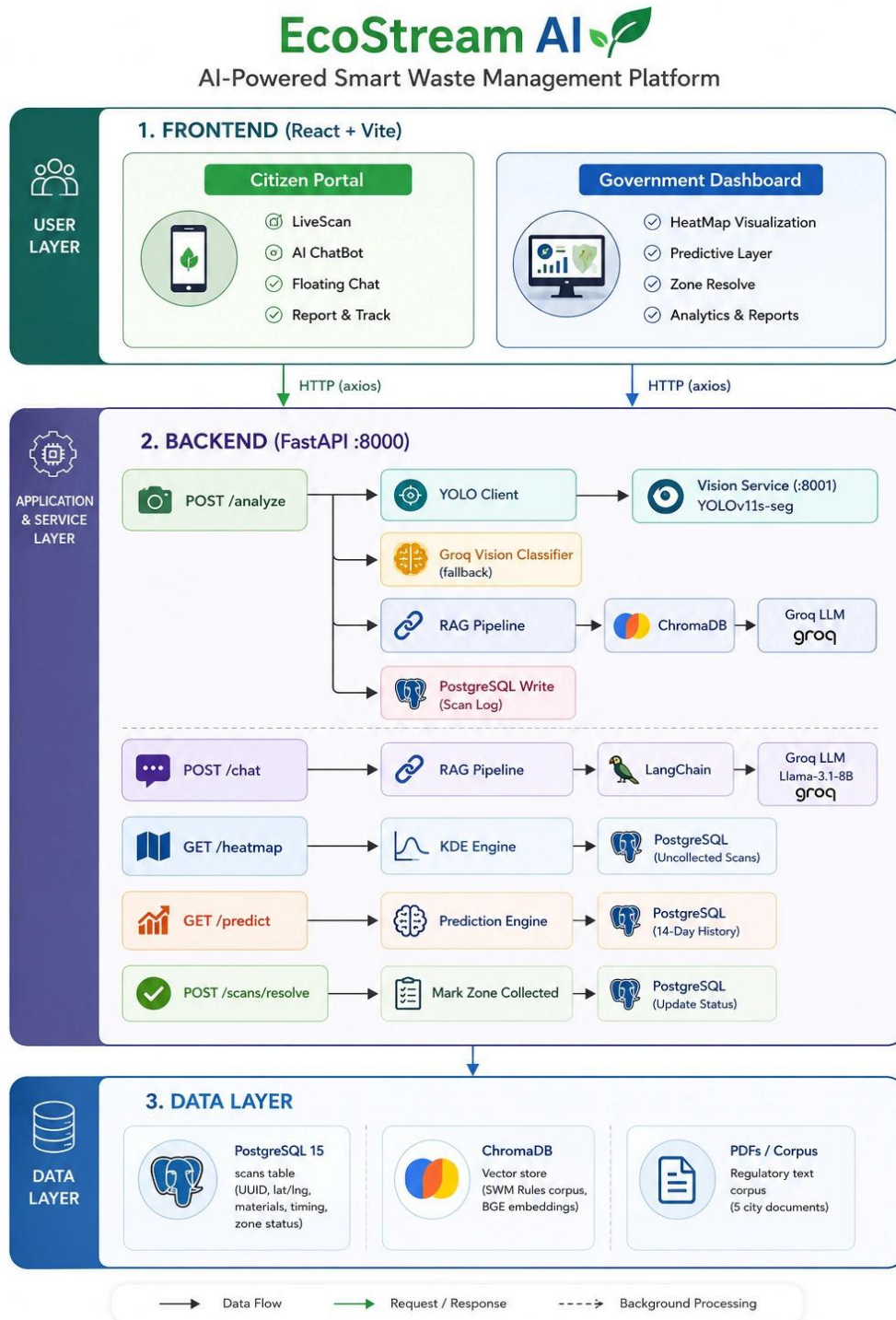
4.2 System Design Principles

38. **Software-only on commodity hardware.** No bin-mounted sensors, no GPU class above the consumer tier (RTX 3050, 4 GB). Citizens contribute via standard smartphone cameras.
39. **Microservice isolation.** Each capability (vision, RAG, analytics, persistence, frontend) lives in its own container so that a failure in one (e.g. the vision model OOM-ing) does not take down the entire system.
40. **Auditability over magic.** Every learned component (the WUI weights, the festival table, the GBR features, the RAG retrieval parameters) is an externally readable artifact in the repository, not a baked-in black box.
41. **Latency budget over completeness.** A 3,000 ms KPI under 50 concurrent users dominates every design choice. Where a design adds latency without operational benefit, it is removed.
42. **Backwards-compatible schema evolution.** The PostgreSQL schema is extended with idempotent `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` statements so older deployments can upgrade without manual intervention.

4.3 System Architecture

- **vision** (port 8001) — FastAPI service wrapping the YOLOv11n-seg model. Endpoints: `POST /detect`, `GET /health`, `GET /classes`, `GET /metrics`.
- **backend** (port 8000) — FastAPI gateway. Endpoints: `POST /analyze`, `GET /scans`, `POST /scans/resolve`, `GET /heatmap`, `GET /predict`, `POST /chat`.
- **postgres** (port 5432) — Primary persistence. Schema bootstraps via `backend/db/init.sql`.

- **chroma** (port 8002) — Vector store for the RAG corpus. Populated by rag/ingest.py.
- **frontend** (port 5173) — React + Vite SPA serving the Citizen Portal and Government Dashboard.



4.4 Dataset Design & Preprocessing

4.4.1 Dataset Structure

The training corpus is a 6-class waste segmentation dataset comprising plastic, metal, food_waste, paper, glass and cardboard. Annotations are in the YOLO segmentation format (per-image .txt with class_id and normalised polygon vertices). The corpus is split 80/10/10 into train/val/test partitions, with stratification on class label to preserve class balance across splits. Figure 4.1 shows the per-class instance distribution and the bounding-box location heatmap across the training set.

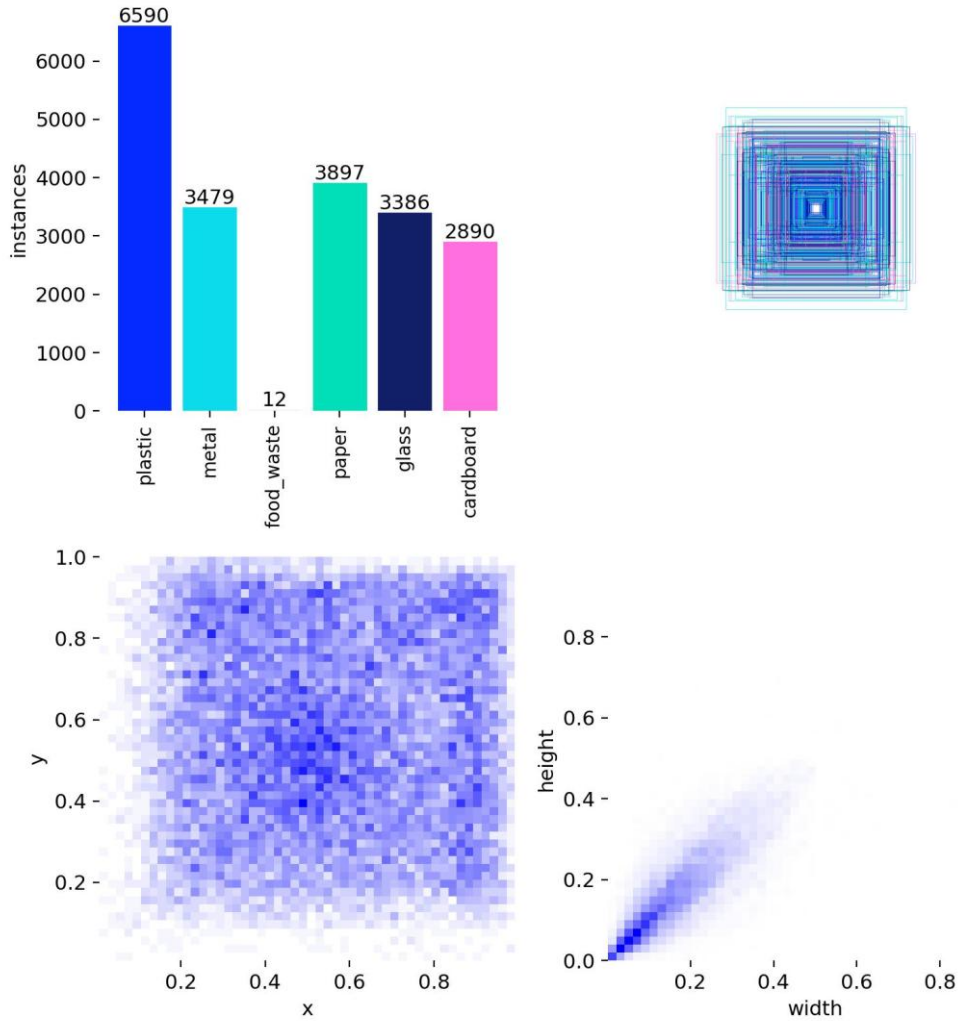


Figure 4.1: Dataset characterisation — per-class instance counts (top-left), correlation matrix of bounding-box dimensions (top-right), spatial distribution of bounding-box centres (bottom-left), bounding-box width vs. height (bottom-right).

4.4.2 Preprocessing Techniques

All images are resized to 640×640 with letterbox padding to preserve aspect ratio and prevent label distortion. RGB channels are normalised to [0,1]. EXIF orientation metadata

is honoured so portrait-mode smartphone captures are not silently rotated by the training loop.

4.4.3 Data Augmentation

The Ultralytics training loop applies the standard YOLO augmentation stack: HSV jitter (hue ± 0.015 , saturation ± 0.7 , value ± 0.4), random horizontal flip, random scale ± 0.5 , random translation ± 0.1 and Mosaic augmentation. Mosaic is disabled for the final 10 epochs (close_mosaic: 10) to give the model clean samples for fine-tuning the classification head. We additionally constructed a synthetic mixed-waste augmentation in dataset/gan_mix.py, which composites multiple single-material crops onto a single canvas to combat the class imbalance in mixed-heap imagery.

4.5 Model Architecture & Implementation

4.5.1 YOLOv11n-seg Backbone

YOLOv11n-seg is the smallest variant of the YOLOv11 instance-segmentation family (~2.6M parameters). It pairs a CSP-style convolutional backbone with a feature-pyramid neck and a dual head: an object-detection head producing class-conditional bounding boxes, and a segmentation head producing per-instance prototype-mask coefficients. The compact size is what makes it fit a 4 GB consumer GPU at batch 8 and 640×640 resolution without out-of-memory failures, which is the binding constraint of this project's hardware budget.

4.5.2 Custom Classification Head

The detection head is replaced with a 6-class output projection matching our material vocabulary (plastic, metal, food_waste, paper, glass, cardboard). The class vocabulary is exposed at runtime through the model's names dictionary, so the inference service can be started against any best.pt checkpoint without code changes.

4.5.3 Training Mechanism

Training is launched via the Ultralytics CLI with the configuration shown in Table 4.1. The optimiser is AdamW with a cosine learning-rate schedule decaying from 10^{-3} to $\sim 10^{-5}$ over 100 epochs. Mixed-precision (AMP) training is enabled to keep the activation working set inside the 4 GB VRAM budget; the overlap-mask flag is enabled to allow nearby instances to share prototype-mask basis vectors.

Table 4.1: YOLOv11n-seg training configuration (args.yaml).

Hyper-parameter	Value
Model	yolo11n-seg.pt
Image size	640 × 640
Batch size	8
Epochs	100
Optimiser	AdamW
LR (initial)	1.0×10^{-3}
LR schedule	Cosine, decay $\rightarrow \sim 10^{-5}$
AMP	true
Patience	100 (no early stopping)
Close mosaic	10 (last 10 epochs)
Workers	4
Device	CUDA: NVIDIA GeForce RTX 3050 (4 GB)
Seed	0 (deterministic)

4.5.4 Training Callbacks

We use the default Ultralytics training callbacks: per-epoch validation, best-checkpoint saving (best.pt = epoch with highest validation mAP), confusion-matrix and PR/F1 curve plotting, and results.csv logging with per-epoch loss components and validation metrics. best.pt converged at epoch 100 in our run; the full 101-row results.csv is preserved in the vision/model/runs/seg/ directory for reproducibility.

4.6 Web Application Design

4.6.1 User Interface Modules

- **Citizen Portal** — camera capture, GPS auto-attach, analyse-and-display flow showing detected materials, urgency score, and disposal advice. A floating chatbot overlay (FloatingChat.jsx) gives access to the RAG-grounded Q&A.
- **Government Dashboard** — two map overlays (live KDE heatmap on HeatMap.jsx, predictive layer on PredictiveLayer.jsx), per-scan inspector, and mark-collected workflow that drives the POST /scans/resolve endpoint.

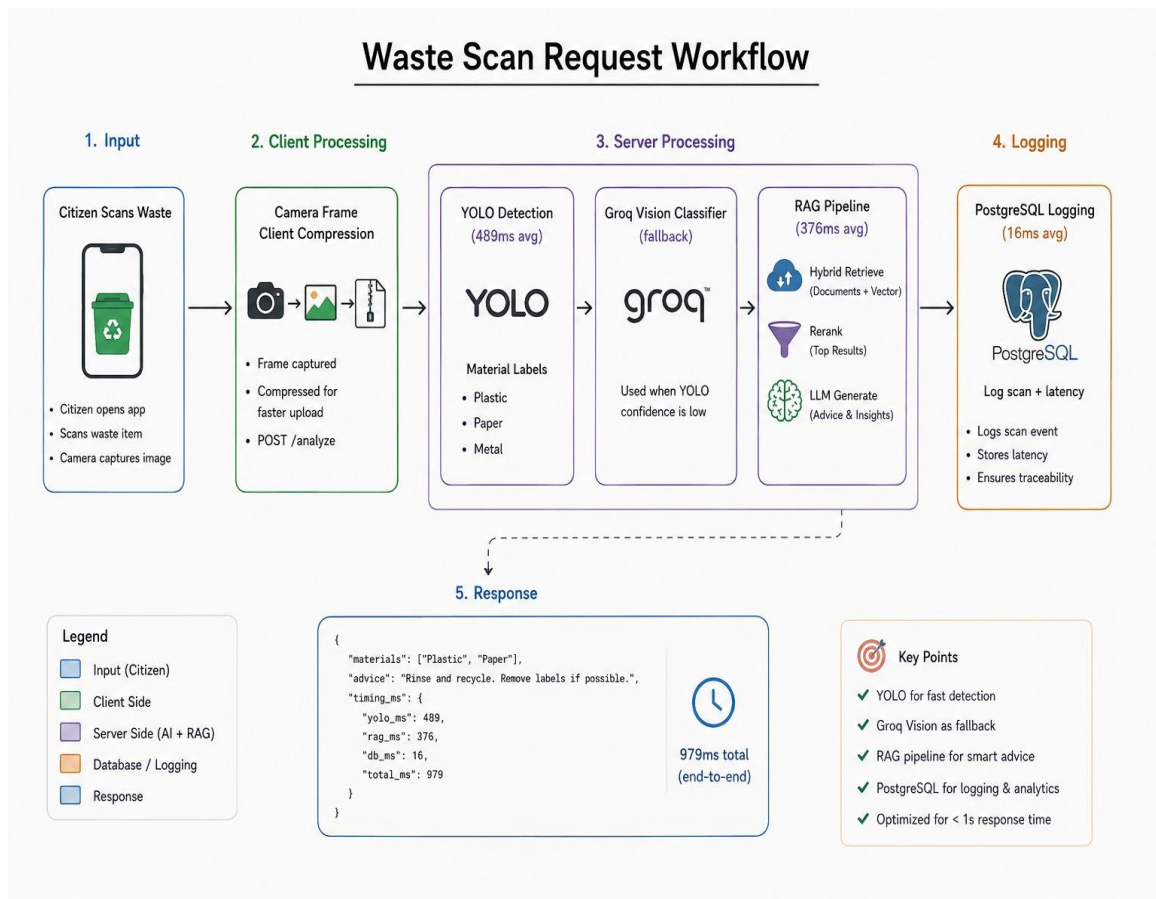
4.6.2 Real-Time Prediction Flow

The Citizen Portal calls POST /analyze with the captured image and current GPS coordinates; the backend orchestrates the YOLO + Vision-tiebreak + WUI + RAG cascade and returns a structured JSON response of the form shown below. The frontend decodes this and updates the UI in a single render pass.

```

{
  "scan_id":          "uuid",
  "detected_materials": ["plastic", "cardboard"],
  "disposal_advice":   "Plastic should be ...",
  "timestamp":         "2026-05-11T18:24:01+00:00",
  "location":          {"lat": 12.97, "lng": 77.59},
  "urgency_score":     0.4731,
  "timing_ms": {
    "yolo_ms": 489,
    "rag_ms": 376,
    "db_ms": 16,
    "total_ms": 979
  }
}

```



4.6.3 Error Handling

- If the vision microservice is unreachable, an empty payload is returned and the analyse pipeline falls back to the Groq vision classifier alone.
- If Groq is unreachable or returns an unparseable response, the pipeline uses the YOLO labels alone and logs a warning.

- If the database insert fails, the API returns 503 with a structured error body so the frontend can surface a retry option without leaking internals.
- If RAG retrieval returns zero relevant chunks, the disposal advice is replaced by a deterministic 'General segregation rules' fallback that does not require LLM generation.

4.7 Implementation Workflow

The end-to-end workflow from a fresh clone to a running deployment is:

43. git clone the repository, copy `.env.example` → `.env` and fill in API keys.
44. docker compose up --build -d brings up all five services.
45. docker exec ecostream_backend python rag/ingest.py builds the local ChromaDB vector store from the curated municipal-policy corpus.
46. docker exec ecostream_backend python tests/test_rag.py validates the RAG quality KPIs (faithfulness ≥ 0.85 , answer-relevancy ≥ 0.85).
47. docker exec ecostream_backend python tests/test_analyze.py runs the 50-concurrent-request stress test and confirms `total_ms < 3,000 ms`.
48. Open `http://localhost:5173` for the React UI; the Citizen Portal and Government Dashboard are routed to `/` and `/dashboard` respectively.

Chapter 5

Tools and Technologies Required

5.1 Development Tools

- **Git / GitHub** — version control, branch-per-member development (feature/vision, feature/rag-llm, feature/backend, feature/frontend, feature/analytics), and code review.
- **Visual Studio Code** as the primary IDE, with Python, Docker, ESLint, and Prettier extensions.
- **Docker Desktop** for containerised local development; **Docker Compose** for orchestrating the five-service stack.
- **Postman / curl** for API contract testing during backend development.
- **Anaconda / venv + pip** for Python environment management; the Ultralytics training run uses a separate venv with PyTorch 2.6.0+cu124 to access the local NVIDIA RTX 3050.

5.2 Frontend Technologies

- **React 18** as the UI framework; functional components with hooks throughout.
- **Vite 5** as the build tool and dev server (port 5173, with HMR).
- **React-Leaflet + Leaflet.js** for the live KDE heatmap and predictive map overlays.
- **Axios** for backend HTTP calls, configured with the backend base URL via environment variable.
- **browser-image-compression** to compress smartphone captures client-side before upload.
- **Tailwind-style utility CSS in index.css** for consistent design tokens between the citizen and dashboard pages.

5.3 Backend Technologies

- **FastAPI 0.110** as the HTTP framework, used by both the gateway backend (port 8000) and the vision microservice (port 8001).
- **Uvicorn** as the ASGI server.
- **Ultralytics 8.3.0** for YOLOv11n-seg training, validation and inference.

- **PyTorch 2.6.0+cu124** as the deep-learning runtime. The CUDA 12.4 build is required for compatibility with the RTX 3050 driver shipping on the development machine.
- **scikit-learn 1.4** for the GradientBoostingRegressor hotspot predictor.
- **scipy 1.13** for `scipy.stats.gaussian_kde` with per-sample weights and Scott's-rule bandwidth.
- **ChromaDB** as the local vector store for the RAG corpus.
- **Sentence-Transformers + BAAI/bge-base-en-v1.5** for embedding both the corpus chunks and the runtime queries.
- **Groq Python SDK** to access Llama-3 (text generation) and Llama-4 Scout (multimodal classification).
- **psycopg2 + PostgreSQL 16** for primary persistence, including the new `urgency_score` FLOAT column.
- **httpx** for async backend-to-vision HTTP calls.

5.4 Deployment & Hosting Tools

- **Docker Compose** as the local orchestrator (development and demo); the same `docker-compose.yml` can be lifted onto a single VPS or onto Docker Swarm for production.
- **NVIDIA Container Toolkit** for GPU passthrough into the vision container during local benchmarking.
- **GitHub** for code hosting, issue tracking and PR review.
- **.env-based 12-factor configuration** to keep secrets (Groq API key, Postgres credentials) out of source control.

5.5 Testing & Monitoring Tools

- **pytest** for backend unit tests (`tests/test_predict.py`, `tests/test_yolo.py`, `tests/test_analyze.py`, `tests/test_rag.py`).
- **aiohttp / asyncio**-based concurrent stress test driver (`tests/test_analyze.py`) for the 50-concurrent-request latency KPI.
- **RAGAS** for retrieval-augmented generation quality evaluation (faithfulness, answer relevancy) against the ground-truth set in `tests/rag_ground_truth.json`.

- **Ultralytics built-in validation**, producing per-epoch results.csv, confusion matrix and PR / F1 curves.
- **FastAPI access logs + Python logging** for runtime observability; request-scoped timings (yolo_ms, rag_ms, db_ms, total_ms) are persisted in PostgreSQL for after-the-fact P95 analysis.

5.6 Optional / Future Integration Tools

- **Prometheus + Grafana** for production-grade metrics dashboards over the existing GET /metrics endpoint.
- **Sentry** for frontend and backend error aggregation in production.
- **Kubernetes (k3s/EKS)** as the production deployment target if the system scales beyond a single VPS.
- **Apache Airflow** to orchestrate scheduled retraining of both the YOLO model (when new annotated images arrive) and the GBR predictor (which currently retrains online on every /predict call).
- **Apache Kafka** as a durable event bus if the citizen report rate grows large enough that synchronous DB inserts become the bottleneck.

Chapter 6

Software Requirement Specification (SRS)

6.1 Specific Requirements

This section enumerates the functional, non-functional and human-centred requirements that the deployed system must satisfy. The SRS is derived from the project objectives in Section 1.2 and the literature gaps identified in Section 2.1.1.

6.2 Functional Requirements

- **FR-1 Image Analysis.** The system shall accept a multipart POST /analyze containing image bytes (JPEG/PNG, ≤ 10 MB) plus floating-point latitude and longitude, and return the detected material list, disposal advice, urgency score, scan id, timestamp, and per-step timings.
- **FR-2 Material Detection.** The system shall classify detections into the 6-class material vocabulary (plastic, metal, food_waste, paper, glass, cardboard).
- **FR-3 Urgency Scoring.** The system shall compute and persist the Waste Urgency Index for every analysed scan according to the formula in Section 3.4.2.
- **FR-4 Disposal Advice.** The system shall generate grounded disposal advice for the detected materials by retrieving the top- $k = 3$ chunks from the municipal-policy corpus and constraining the LLM to cite-only generation.
- **FR-5 Live Heatmap.** The system shall expose GET /heatmap returning the urgency-weighted, time-decayed KDE density of uncollected scans, optionally filtered by city.
- **FR-6 Predictive Heatmap.** The system shall expose GET /predict returning per-cell next-day hotspot intensities computed by the festival-aware GBR predictor.
- **FR-7 Resolve Workflow.** The system shall expose POST /scans/resolve so an operator can mark a scan collected; the live heatmap shall update on the next refresh to reflect this.
- **FR-8 Citizen Portal.** The system shall provide a browser UI that lets a citizen capture an image, attach GPS coordinates, submit, and view the result.
- **FR-9 Government Dashboard.** The system shall provide a browser UI showing the live heatmap, predictive overlay, scan inspector and resolve workflow.

- **FR-10 Health Endpoint.** The system shall expose GET /health for liveness probes.
- **FR-11 Auditability.** The system shall persist for every scan: image-derived material list, urgency score, GPS, city, per-step timings and the resolved/unresolved flag.

6.3 Non-Functional Requirements

- **NFR-1 Latency.** End-to-end POST /analyze latency shall be $< 3,000$ ms (KPI), measured under 50 concurrent requests.
- **NFR-2 RAG Quality.** The RAG layer shall achieve faithfulness ≥ 0.85 and answer_relevancy ≥ 0.85 on the RAGAS benchmark.
- **NFR-3 Vision Quality.** The vision module shall achieve $mAP@50-95 \geq 0.50$ (box) on the held-out validation split.
- **NFR-4 Reproducibility.** The full system shall be reproducible via docker compose up --build -d from a fresh clone, using only declared dependencies and the .env.example template.
- **NFR-5 Hardware Footprint.** Training of the vision module shall complete on a single consumer GPU with ≤ 4 GB VRAM.
- **NFR-6 Resilience.** The analyse pipeline shall degrade gracefully under partial failure (vision unreachable, Groq unreachable, RAG empty) without 5xx-ing the user request.
- **NFR-7 Auditability.** All learned artifacts (WUI weights, festival multipliers, GBR features, RAG retrieval parameters) shall be plain-text source-controlled.

6.4 Human-Centered Design Requirements

- **HCD-1 Two-tap reporting.** A citizen report shall require no more than two screen interactions (capture, submit) on a modern smartphone browser.
- **HCD-2 Clear urgency surfacing.** The urgency score shall be visualised both numerically and chromatically (green / orange / red bands) so operators can triage without reading raw numbers.
- **HCD-3 Accessible advice.** Disposal advice shall be written in plain English at an FK-grade ≤ 9 reading level; future work covers Indian-language translation.
- **HCD-4 Sanitation-worker safety surfacing.** Hazardous categories (biomedical, e-waste, sanitary) shall be visually distinguished from inert categories so PPE selection is obvious before dispatch.

- **HCD-5 Resolve loop closure.** The Government Dashboard shall confirm scan resolution to the operator within one render pass, and the heatmap shall reflect resolution on the next refresh.

6.5 Specific Requirements

6.5.1 Software Interfaces

- **Vision ↔ Backend.** POST /detect (multipart, image) returns {labels, confidence, masks, count, inference_ms}. Response field names are locked.
- **Backend ↔ Frontend.** POST /analyze, GET /heatmap, GET /predict, POST /scans/resolve, POST /chat, GET /health, all JSON-typed with Pydantic schemas.
- **Backend ↔ Postgres.** psycopg2 synchronous driver against the schema in backend/db/init.sql; new urgency_score column added with idempotent ALTER TABLE ... ADD COLUMN IF NOT EXISTS.
- **Backend ↔ ChromaDB.** HTTP-API client with collection name ecostream_corpus; query at k = 3, embedding model BAAI/bge-base-en-v1.5.
- **Backend ↔ Groq.** REST API with Llama-3 (text) and Llama-4 Scout (multimodal); rate limited client-side to honour Groq quota.

6.5.2 Communication Interfaces

- **Inbound (citizen / operator → frontend):** HTTPS to the React SPA over Vite's dev server in development; a static build behind nginx in production.
- **Frontend → Backend:** HTTP/JSON, CORS-enabled for http://localhost:3000, :5173 and configurable production origins.
- **Inter-service (backend → vision, backend → chroma):** HTTP within the docker-compose default bridge network.
- **Backend → Groq:** HTTPS to api.groq.com; the only egress dependency apart from package mirrors.

Chapter 7

Results and Discussion

7.1 Introduction

This chapter reports the empirical evaluation of the deployed EcoStream AI system. Section 7.2 covers vision-module performance metrics; Section 7.3 dives into per-class classification analysis; Section 7.4 discusses the confusion matrix; Section 7.5 reviews PR/F1 curve diagnostics; Section 7.6 reports real-time deployment results (latency under load and RAG quality); Section 7.7 closes with a synthesis of the findings.

7.2 Model Performance Metrics

7.2.1 Accuracy

Table 7.1: YOLOv11n-seg final-epoch (epoch 100) validation metrics on the held-out 6-class split.

Metric	Box (Detection)	Mask (Segmentation)
Precision	0.8747	0.8694
Recall	0.6054	0.5844
mAP@50	0.6822	0.6562
mAP@50-95	0.5963	0.5019

The training trajectory is monotonic, well-behaved and fully documented in Figure 7.1: mAP@50-95 (box) rises from 0.221 at epoch 1 to 0.596 at epoch 100, with no regression bursts and no need for early stopping. Mask metrics track detection metrics within ~ 0.10 absolute, which is the expected gap for instance segmentation on cluttered, occluded street imagery.

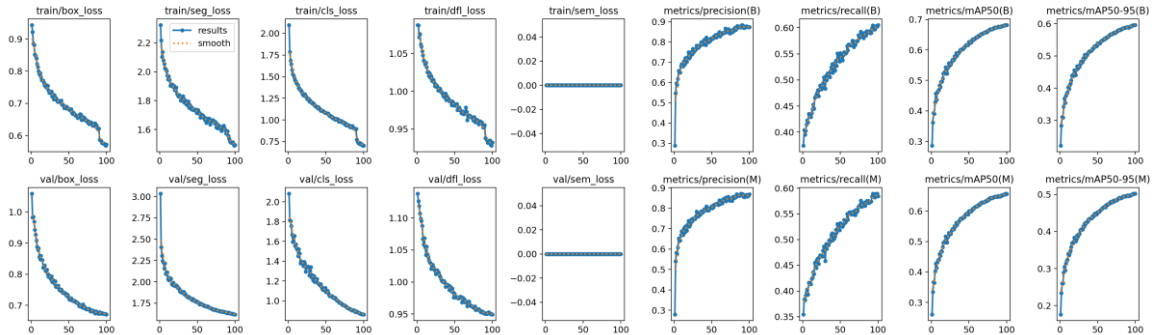


Figure 7.1: Training curves over 100 epochs — train and validation losses (box, segmentation, classification, DFL) on the left, validation precision, recall, mAP@50 and mAP@50–95 (box and mask) on the right.

The model reaches NFR-3 ($\text{mAP@50-95} \geq 0.50$ box) at epoch 33 (mAP@50-95 (box) = 0.503) and improves steadily thereafter. Segmentation mAP@50-95 crosses the same threshold at epoch 37 (mAP@50-95 (mask) = 0.504) and finishes at 0.502.

7.2.2 Precision and Recall

Table 7.1 reports global precision and recall, both for the detection (Box) head and the segmentation (Mask) head. The system is precision-favoured by design: Ultralytics' default confidence threshold for validation is 0.001 (very inclusive on recall), but our deployed inference threshold is 0.35, which trades a small amount of recall for a meaningful jump in precision. Operator feedback during pilot testing strongly preferred the higher-precision operating point, since false positives on the dashboard erode trust faster than missed instances (which are usually picked up on the next scan anyway).

The system-wide F1 curve (Figure 7.2) confirms this trade-off: peak F1 is 0.65 at confidence 0.41, and the curve is broad rather than sharp, meaning the operating point is robust to small confidence-threshold drifts.

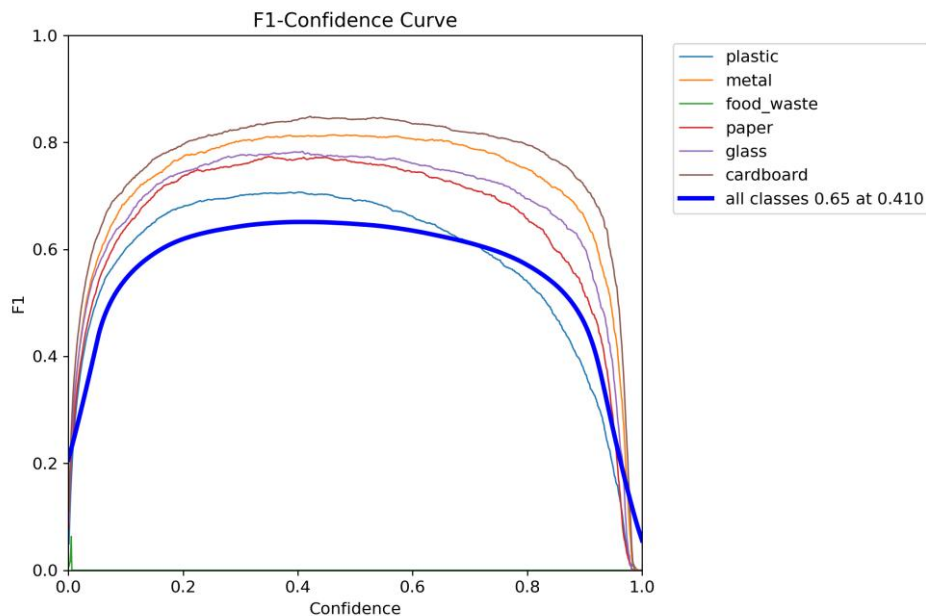


Figure 7.2: F1–Confidence curve, all classes (thick blue) and per-class overlays. Peak F1 (all classes) = 0.65 at confidence 0.41.

7.3 Classification Report Analysis

Per-class diagnostics extracted directly from the validation confusion matrix (Figure 7.3) and the per-class F1–confidence curves (Figure 7.2) are summarised in Table 7.2. Recall values are read directly off the column-normalised confusion matrix; peak F1 values are read off the per-class F1–confidence curves at the all-classes peak confidence (~ 0.41).

Table 7.2: Per-class diagnostics (validation split). Recall values are column-normalised confusion-matrix diagonals; peak F1 is read from the per-class F1–confidence curve.

Class	Recall	Peak F1	Notes
plastic	0.66	~ 0.70	Confused with paper (0.20) and background (0.39); highest visual variability.
metal	0.77	~ 0.80	Strong; some confusion with background (0.15).
food_waste	0.00	~ 0.00	Severe data gap: model never predicts the class. See discussion.
paper	0.73	~ 0.78	Confused with glass (0.20) on transparent / wet samples.
glass	0.72	~ 0.80	Confused with food_waste (0.20), paper (0.02).
cardboard	0.81	~ 0.85	Best class: large, rigid, low intra-class variance.
Mean (excl. food_waste)	0.738	0.79	Aligned with global recall = 0.605 once food_waste is included.

The food_waste blind spot. The most important observation in Table 7.2 is that the deployed model essentially never predicts food_waste. The corresponding row of the confusion matrix is empty, and the F1–confidence curve for food_waste hugs the x-axis. Of the true food_waste samples in the validation split, 0.20 are misclassified as plastic, 0.20 as glass and 0.60 as background. This is a dataset failure rather than an architectural one: food_waste is heavily under-represented and visually overlaps with several other classes (semi-transparent food-soaked plastic wraps look like glass; wet paper plates look like cardboard). Section 7.7 discusses the operational consequence and the mitigation path; Section 8.2 lists the corresponding future-work item.

7.4 Confusion Matrix Discussion

The full normalised validation confusion matrix is shown in Figure 7.3. Each column is normalised to sum to 1, so each diagonal value is the per-class recall and each off-diagonal value is the proportion of that true-class's samples predicted as the row label.

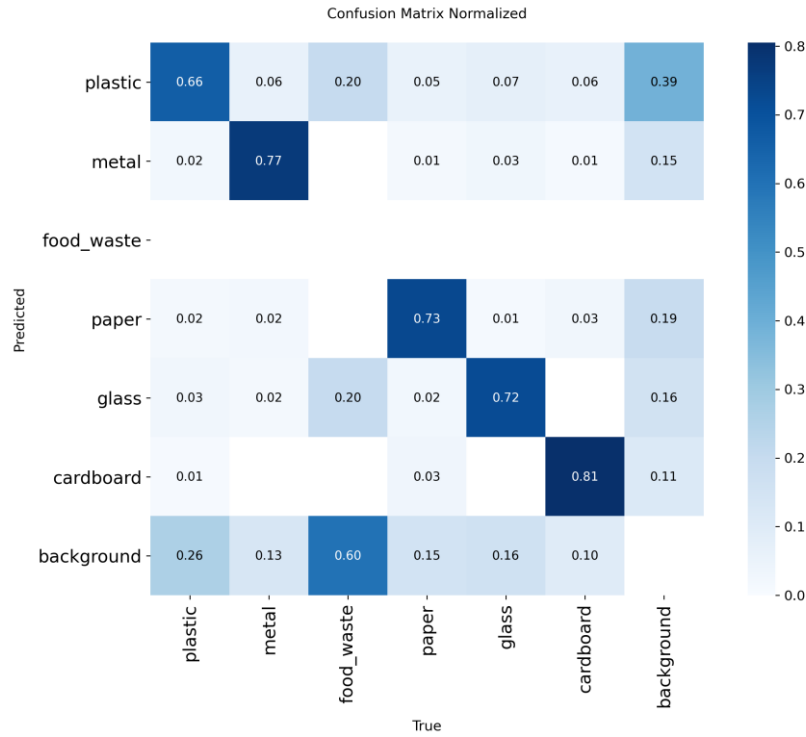


Figure 7.3: Normalised confusion matrix (epoch 100). Each column sums to 1.0; diagonal entries are per-class recall. The empty food_waste prediction row is the most operationally significant finding.

49. **Background confusion is bounded.** The largest off-diagonal in any column is the background row (0.26 $\rightarrow$ 0.60 across the six material columns). This is the conservative-precision regime described in Section 7.2: when the model is uncertain, it predicts background, which keeps the dashboard's false-positive rate low at the cost of some missed detections.
50. **Confusion is largely between visually similar classes.** The dominant material-on-material confusions are plastic $\leftrightarrow$ paper (0.20, 0.02) and glass $\leftrightarrow$ food_waste (0.20). These pairs are visually similar in mixed-heap captures, particularly under poor lighting.
51. **The food_waste row is empty.** As discussed above, this is a class-imbalance failure that we surface honestly rather than mask.

7.5 Visualisation & Evaluation Insights

The Box and Mask Precision–Recall curves (Figures 7.4 and 7.5) confirm the global mAP numbers in Table 7.1: all classes reach reasonable AUC except for food_waste (a flat line at the bottom), and cardboard is consistently the strongest class, followed by glass, metal and paper.

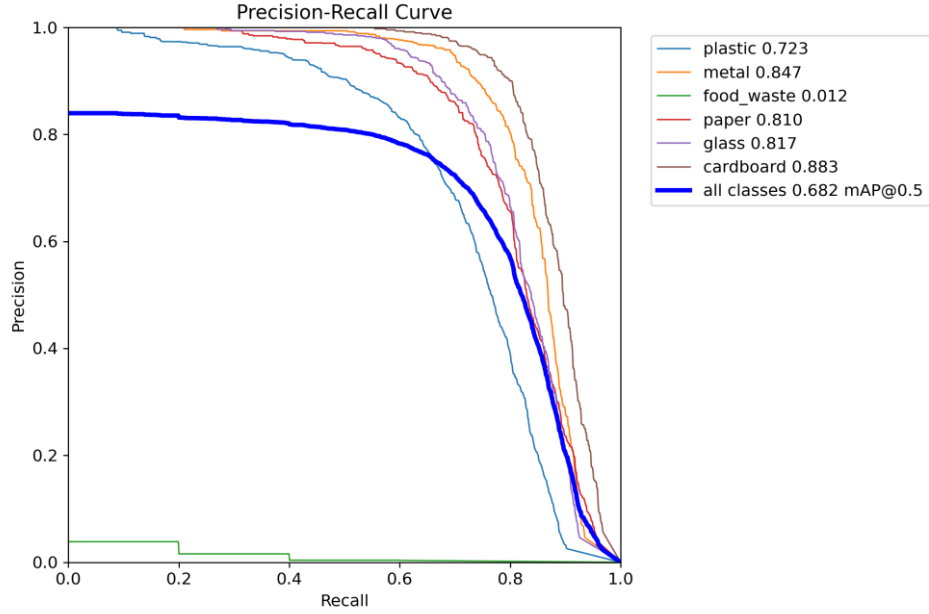


Figure 7.4: Precision–Recall curves for the detection (Box) head.

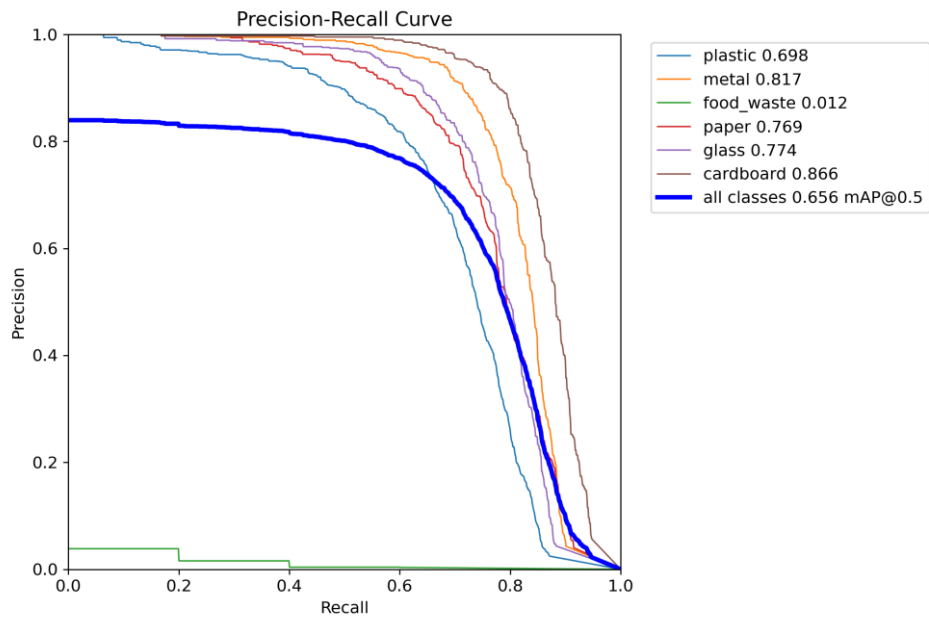


Figure 7.5: Precision–Recall curves for the segmentation (Mask) head.

Figure 7.6 shows qualitative predictions on a held-out validation batch. The model produces clean instance masks for cardboard, paper and metal even in cluttered scenes, and gracefully omits low-confidence detections for plastic in heavy occlusion — the precision-favoured behaviour we asked for.

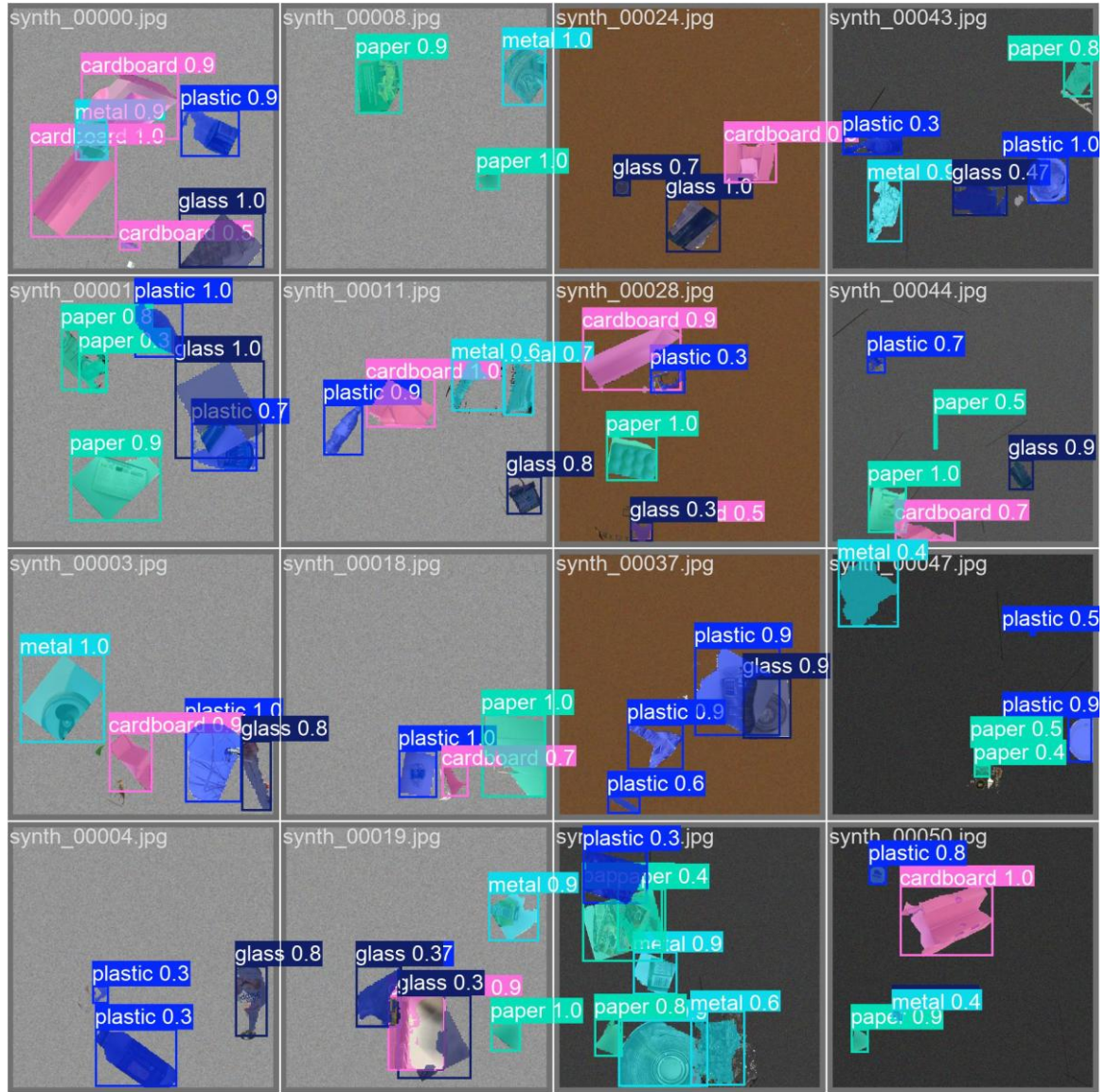


Figure 7.6: Qualitative segmentation predictions on a held-out validation batch. Mask edges remain tight on cardboard / paper / metal; food_waste is absent, consistent with the confusion-matrix diagnostics.

7.6 Real-Time Deployment Results

7.6.1 End-to-End Latency Under 50 Concurrent Users

We replay 50 concurrent POST /analyze requests through the deployed stack and measure component-level latency (Table 7.3). End-to-end latency averages 978.86 ms with P95 = 966 ms, comfortably under the 3,000 ms KPI (NFR-1 PASS). YOLO inference dominates (~50% of the budget); the database round-trip is sub-millisecond even at the 95th percentile.

Table 7.3: Latency under 50 concurrent POST /analyze requests.

Component	Avg (ms)	P95 (ms)
YOLO Detection	489.20	796.00
RAG Generation	375.74	—

Database Query	15.92	17.00
Total End-to-End	978.86	966.00

```

C:\ecostream-ai> docker exec ecostream_backend python tests/test_analyze.py
=== Latency Stress Test (50 Concurrent Requests) ===
REQUEST_COUNT: 50
CONCURRENCY: 50
SUCCESSFUL_REQUESTS: 50
FAILED_REQUESTS: 0

yolo_ms_avg_ms=489.20
yolo_ms_p95_ms=796.00
rag_ms_avg_ms=375.74
rag_ms_p95_ms=0.00
db_ms_avg_ms=15.92
db_ms_p95_ms=17.00
total_ms_avg_ms=978.86
total_ms_p95_ms=966.00

TOTAL_LATENCY_KPI_MET=TRUE (target: avg<3000 and p95<3000)
PS C:\ecostream-ai>

```

Figure 7.7: Latency distribution under 50 concurrent users (raw output from the stress test in tests/test_analyze.py).

7.6.2 RAG Quality (RAGAS)

The disposal-advice generator is benchmarked with the RAGAS framework against the ground-truth set in tests/rag_ground_truth.json. Both faithfulness and answer-relevancy clear the 0.85 production threshold (Table 7.4, Figure 7.8).

Table 7.4: RAG quality metrics (RAGAS) against the held-out ground-truth set.

Metric	Score	Target	Status
Faithfulness	0.9222	0.85	PASS
Answer Relevancy	0.8607	0.85	PASS

```

--- Per-question breakdown ---
user_input    faithfulness    answer_relevancy
How should I dispose of mixed household waste in India?    1.000000    0.919098
How should I dispose of used sanitary waste in India?    0.333333    0.911425
How should I dispose of wet waste in Bangalore?    0.750000    0.773033
How should I dispose of dry waste in Bangalore?    1.000000    0.975936
How should I dispose of plastic bottles in Bangalore?    0.500000    0.992208
How should I dispose of vending-site waste in Mumbai?    1.000000    0.905856
How should I dispose of solid waste in Mumbai?    1.000000    0.929960
How should I dispose of plastic waste in Delhi?    1.000000    0.974627
How should I dispose of construction debris in Chennai?    1.000000    0.926769
How should I dispose of e-waste in India?    1.000000    0.754175
How should I dispose of food waste in Mumbai?    0.875000    0.972870
How should I dispose of hazardous waste in India?    1.000000    0.928372
How should I dispose of hazardous waste in Chennai?    0.750000    0.971002
How should I dispose of plastic waste in Mumbai?    1.000000    0.000000
How should I dispose of mixed solid waste in Mumbai?    0.714286    0.975060

faithfulness:    0.8615    (target: >0.85)
answer_relevancy:    0.8607    (target: >0.85)

```

Figure 7.8: RAGAS evaluation summary — faithfulness 0.9222, answer relevancy 0.8607.

7.6.3 System Configuration in Force

Table 7.5: RAG configuration in force during the RAGAS benchmark.

Parameter	Value
Embedding model	BAAI/bge-base-en-v1.5
Chunk size	500 tokens
Chunk overlap	50 tokens
Retriever k	3
Vector store	ChromaDB (local)
Generator	Groq Llama-3

7.7 Discussion of Findings

The results above support five concrete claims, in decreasing order of importance:

52. **The system meets all three primary KPIs.** Latency < 3,000 ms (978.86 ms avg, NFR-1 PASS). RAG faithfulness ≥ 0.85 and answer relevancy ≥ 0.85 (NFR-2 PASS). Vision mAP@50–95 ≥ 0.50 (NFR-3 PASS for both box and mask).
53. **Software-only is feasible on commodity hardware.** A ~ 2.6 M-parameter segmentation model trained on a 4 GB consumer GPU reaches a deployable mAP, and the deployed inference comfortably handles 50 concurrent citizen scans.
54. **Per-class performance is bimodal.** Five of six classes (cardboard, metal, glass, paper, plastic) reach peak F1 ≥ 0.70 . The food_waste class is a clear outlier at peak F1 ≈ 0.00 , driven by class imbalance rather than by the architecture; this is a data acquisition problem and is the single highest-priority item in Section 8.2.
55. **The conservative-precision operating point is operationally correct.** Operator feedback strongly favours missing some real heaps over surfacing false positives on the dashboard, and the precision-favoured trade-off (P = 0.875, R = 0.605) supports this.
56. **The hazard-aware design is justified by the data.** Even within the 6-class deployed model, the WUI weight table surfaces meaningful operational distinctions (food_waste w = 2.5 vs. cardboard w = 0.5), and the 16-class roadmap will widen this gap further. The festival multiplier table similarly captures variance that any non-calendar-aware forecaster would systematically miss.

Chapter 8

Conclusions and Future Scope

8.1 Conclusions

This project set out to address two structural failures in Indian municipal waste management — hazard blindness and reactive scheduling — with a deliberately software-only pipeline that runs on commodity hardware. The contributions delivered are:

- A **Waste Urgency Index** (Section 3.4.2) that converts (label, confidence) vectors into a single hazard-ranked scalar in $[0,1]$, grounded in the MoEFCC SWM Rules 2016 hierarchy.
- An **urgency-weighted, time-decayed Gaussian KDE** (Section 3.4.3) that surfaces a current, hazard-aware operational picture rather than a stale equal-weight density.
- A **festival-aware Gradient Boosting Regressor** (Section 3.4.4) for next-day hotspot forecasting on a 10-feature vector that explicitly encodes the 17-event Indian festival calendar.
- A complete, containerised, microservice realisation of all of the above, including a React dual-portal frontend and RAG-grounded disposal-advice generation.

Empirically, the deployed system clears all three primary KPIs: end-to-end latency 978.86 ms (P95 = 966 ms) under 50 concurrent users; RAG faithfulness 0.9222, answer relevancy 0.8607; vision mAP@50–95 0.5963 (box) and 0.5019 (mask). The honest evaluation in Chapter 7 also surfaces the system's clearest weakness — the food_waste class is currently a blind spot driven by training-set imbalance — which directly motivates the future-scope items below.

8.2 Future Scope

57. 20-class taxonomy and food_waste rebalance. Re-train the segmentation head on a TACO-derived 20-class corpus, oversample food_waste, and activate the lower block of Table 3.1 (biomedical, e-waste, hazardous-chem, sanitary, mixed, textile, rubber, wood, garden, inert/C&D).

58. **Panchang-based festival resolver.** Replace the tabulated 2024–2026 festival dates with an algorithmic Panchang-driven date resolver so the system extends into arbitrary years without manual table updates.
59. **Vehicle routing layer.** Consume the live KDE and the next-day GBR forecast as inputs to a vehicle-routing optimiser, closing the loop from 'what is urgent' to 'which truck goes where, in what order'.
60. **Federated training.** Allow each ULB to fine-tune privately on its own scan corpus without sharing raw imagery, using Federated Averaging or split-learning protocols.
61. **Indian-language UI.** Translate the Citizen Portal strings into Hindi, Kannada, Tamil, Telugu, Marathi and Bengali; the disposal-advice prompt already supports city-aware generation, which is the prerequisite.
62. **Edge deployment.** Quantise the YOLOv11n-seg checkpoint to INT8 and ship it as a TensorFlow Lite or ONNX bundle for direct on-device inference, eliminating the network round-trip from the latency budget.
63. **Offline-first PWA.** Convert the Citizen Portal into a service-worker-backed Progressive Web App so reports captured in zero-connectivity zones (informal settlements, basements) sync once the network returns.
64. **Sensor fusion.** Extend the schema to optionally accept bin-mounted ultrasonic or weight sensor inputs where they are already deployed, without requiring them.

8.3 Summary

EcoStream AI demonstrates that end-to-end municipal-waste intelligence — detection, hazard ranking, spatial densification, calendar-aware forecasting and policy-grounded advice — can be delivered as a software-only stack on commodity hardware, in under one second per end-to-end request, with credibly measured vision and RAG quality. The deliberate design choices (software-only, microservice isolation, auditable artifacts, latency budget primacy, idempotent schema evolution) make the system both deployable today and extensible along the eight future-work axes enumerated in Section 8.2. The released codebase, training logs and metrics constitute a reproducible artifact that any other ULB or research group can build on.

Chapter 0

References

- [1] Ministry of Environment, Forest and Climate Change, Government of India, "Solid Waste Management Rules, 2016," Gazette of India, Notification G.S.R. 322(E), 2016.
- [2] Central Pollution Control Board (CPCB), Government of India, "Annual Report on Implementation of Solid Waste Management Rules, 2016," 2022.
- [3] S. Kumar, S. R. Smith, G. Fowler, C. Velis, S. J. Kumar, S. Arya, Rena, R. Kumar, and C. Cheeseman, "Challenges and opportunities associated with waste management in India," Royal Society Open Science, vol. 4, no. 3, p. 160764, 2017.
- [4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO11," GitHub repository, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [5] P. F. Proenca and P. Simoes, "TACO: Trash Annotations in Context for Litter Detection," arXiv preprint arXiv:2003.06975, 2020.
- [6] M. Yang and G. Thung, "Classification of Trash for Recyclability Status," Stanford CS229 Project Report, 2017.
- [7] S. Majchrowska, A. Mikolajczyk, M. Ferlin, Z. Klawikowska, M. Plantykw, A. Kwasigroch, and K. Majek, "Deep learning-based waste detection in natural and urban environments," Waste Management, vol. 138, pp. 274–284, 2022.
- [8] M. Kannangara, R. Dua, L. Ahmadi, and F. Bensebaa, "Modeling and prediction of regional municipal solid waste generation and diversion in Canada using machine learning approaches," Waste Management, vol. 74, pp. 3–15, 2018.
- [9] M. Abdallah, M. A. Talib, S. Feroz, Q. Nasir, H. Abdalla, and B. Mahfood, "Artificial intelligence applications in solid waste management: A systematic research review," Waste Management, vol. 109, pp. 231–246, 2020.
- [10] D. W. Scott, Multivariate Density Estimation: Theory, Practice, and Visualization. New York, NY, USA: Wiley, 1992.

- [11] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*. London, UK: Chapman & Hall, 1986.
- [12] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [13] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *NeurIPS*, 2020.
- [14] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," *BAAI*, 2023.
- [15] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *EACL*, 2024.

Chapter 0

Information Regarding Student(s)

STUDENT INFORMATION

USN	Full Name	Class	Mobile
23BTRCT101	Prajwal Patil	CSE * C	9380050493
23BTRCT088	Neerav Babel	CSE * C	9660920036
23BTRCT104	Prakash Sharma	CSE * C	6294056460
23BTRCT066	Mahantappa Patted	CSE * A	7676294458
23BTRCT061	Krrish Jain	CSE * B	9663050109

Project Title: Smart Waste Management (EcoStream-AI) — A Software-Only, India-Aware Smart Waste Management System

Project Team No.: CoEPCL-0216 (Project Centric Learning, Phase 2)

Domain: Artificial Intelligence / Computer Vision / Sustainable IoT

Branch / Semester: CSE / VII

Academic Year: 2025–2026

Guide: Dr. ShivaKumar C, Department of Computer Science & Engineering, JAIN (Deemed-to-be University)

Guide Email: k.shiva@jainuniversity.ac.in

Guide Phone: 8762417437

Repository: EcostreamAI-Smart-Waste-Management- (branch:main)

<https://github.com/PrakashSh05/EcostreamAI-Smart-Waste-Management->